When 0.99 Becomes 0.67: A Confirmatory Failure of Lexical Policy-Impact Prediction

Chih-Kai Wang [1]

Abstract

Policy-governed agents need to identify which compiled guards become stale when a natural-language policy changes. We study a lightweight RuleDiff baseline that predicts affected guards by thresholding inverse-document-frequency weighted lexical cosine similarity. A 12-configuration adaptive search on an airline development benchmark selected a threshold of 0.20 and reached stratified macro-F1 0.9919. We froze this configuration and evaluated it once on a separately constructed retail-policy packet, where performance fell to 0.6661 (95% source-block bootstrap interval 0.5795–0.7152), a 0.3258 absolute decrease. The private packet contained 14 affected-guard false negatives; its sole high-fan-out tightened-prohibition case missed all five affected guards, although this single confounded case cannot establish a general fan-out effect. The result motivates a layered governance architecture in which selective impact prediction is only an optimization, while an independently implemented current-policy check and fail-closed revision handling remain authoritative at the action boundary.

1 Introduction

Agent systems increasingly compile natural-language organizational policy into executable checks around tools. ToolGuard, for example, maps policy fragments to tools offline and executes generated validators before invocation (Zwerdling et al., 2025). This architecture raises a lifecycle question that is distinct from initial guard generation: after a policy revision, which existing guards must be invalidated, regenerated, or reviewed?

A cheap answer is lexical impact prediction. If a rewritten clause and a guard description share rare operational terms, their similarity can serve as an invalidation hint. Such a mechanism is attractive because it is deterministic, interpretable, and inexpensive. It is also dangerous to trust without held-out evidence: a cross-cutting requirement can alter multiple behaviors without repeating each guard's local vocabulary.

We evaluate this risk through a staged research contract. Development uses airline policies and ToolGuard-derived guard descriptions; confirmatory evaluation uses a fresh retail-policy packet constructed only after model selection. The selected inverse-document-frequency (IDF) cosine model scores 0.9919 on development but only 0.6661 on the single private run. We treat this decrease as the main result rather than hiding it behind the development score.

The paper makes three evidence-bounded contributions. First, we specify a reproducible policy-impact classification task with source-clause-block uncertainty and asymmetric operational interpretation of false negatives. Second, we report a frozen cross-domain confirmatory result: private stratified macro-F1 falls by 0.3258, with 14 stale false negatives. Third, we extract conservative system design requirements from that failure: prediction can prioritize recompilation, but it cannot authorize continued execution; action-boundary checks need independent policy currency evidence and fail-closed behavior. The first two are measured results. The third is a design implication, not an experimentally validated end-to-end safety guarantee.

2 Problem and Method

2.1 Policy-impact prediction

Let $A = \{a_j\}$ be compiled guard artifacts and let $r_i = (c_i, c_i')$ be a policy transition from an old clause to a rewritten clause. The label $y_{ij} = 1$ means that recompiling the policy changes the expected behavior of artifact a_j. Every transition belongs to a policy-shift

[1]National Taipei University of Education, Taipei, Taiwan. Correspondence to: Chih-Kai Wang <f0909172434@gmail.com>.

Technical report, September 2026. Not peer reviewed.

class and has impact fan-out $f_i = \sum_j y_{ij}$. We predict

$$\hat{y}_{ij} = \mathbb{1}[s(a_j, c_i \| c_i') \geq \tau], \tag{1}$$

where s is a lexical similarity function and τ is a fixed threshold.

The selected model tokenizes lowercase alphanumeric words, removes a fixed 22-word stop list, assigns smoothed IDF weights $w(t) = \log((1+N)/(1+n_t))+1$, and computes weighted cosine similarity over token sets. No language model, embedding service, or training corpus is used. The development search also tests word Jaccard, character n-gram Jaccard, and convex hybrids.

2.2 Metrics and error semantics

For each observed stratum, defined by shift class and fan-out, we compute binary macro-F1 by averaging the affected and unaffected class F1 values. The headline stratified macro-F1 is the unweighted mean across observed strata. This prevents the many unaffected transition–artifact pairs from dominating the result.

We call $y = 1, \hat{y} = 0$ a stale false negative: the predictor would fail to nominate an affected guard for invalidation. This term names a classification outcome, not an observed stale execution or unsafe database commit. A false positive unnecessarily invalidates an unaffected guard. Both matter, but false negatives are the critical reason that prediction cannot be the final authorization layer.

2.3 Staged protocol

The development packet contains 18 airline guard artifacts and 24 transitions derived from eight source clauses, yielding 432 pairs. For each source clause, annotators construct one tightened prohibition, one relaxed permission, and one conditional exception. A fixed budget permits 12 adaptive configurations. The best development configuration is frozen before confirmatory scoring.

The private packet changes both policy domain and artifact construction. It contains 12 retail guard artifacts and 18 transitions from six source clauses, yielding 216 pairs. Each source clause again contributes all three rewrite classes. Public transitions and private labels are stored separately before scoring. The one-shot evaluator asserts the frozen configuration, refuses to overwrite an existing result, records input hashes, and omits pair labels from its aggregate output.

Confidence intervals resample complete source-clause blocks within class/fan-out signatures. We use 2,000 percentile draws with seed 20260929. With only six private blocks, and only one block carrying the high-fan-out signature, the interval quantifies limited block resampling variation; it does not estimate uncertainty over a population of high-fan-out clauses.

Table 1. The frozen development winner does not transfer at its development performance level. Confidence intervals are percentile source-block bootstrap intervals.

Quantity	Development	Private
Artifacts	18	12
Transitions	24	18
Source blocks	8	6
Pairs	432	216
Stratified macro-F1	0.9919	0.6661
95% interval	[0.9919, 0.9919]	[0.5795, 0.7152]
Pooled macro-F1	0.9923	0.7211
False negatives	0	14
False positives	1	3

3 Experimental Results

3.1 Development selection

Table 1 reports the contract boundaries and headline results. The initial word-Jaccard baseline scores 0.5700. IDF cosine at threshold 0.15 reaches 0.9068, and threshold calibration selects 0.20 at 0.9919. Thresholds 0.18 and 0.25 score 0.9864 and 0.9017, respectively, bracketing the tested development optimum. These are 12 configurations on the same packet, not 12 independent random-seed replications.

The development interval is degenerate because its sole error pattern is invariant under the available resamples. It is not evidence of zero population uncertainty. Selection over a small, fixed benchmark further makes the point estimate optimistic.

3.2 Confirmatory generalization gap

The frozen winner scores 0.666090 on the private packet, with a 95% bootstrap interval of 0.579507–0.715212. Private minus development is −0.325841, a 32.85% relative decrease from the development score. The development point estimate lies outside the private interval, but the interval is for private performance rather than for the paired difference between domains. Because domain, artifact construction, and clause composition all change together, the result establishes failure to transfer to this retail packet; it does not isolate which shift caused the decrease.

The private confusion counts are 8 true positives, 191 true negatives, 3 false positives, and 14 false negatives.

Table 2. Private confusion counts by observed stratum. The high-fan-out row is one transition, not a replicated fan-out experiment.

Shift, fan-out	TP	FN	FP	TN	F1
Conditional, 1	4	2	0	66	0.893
Relaxed, 1	2	4	1	65	0.704
Tightened, 1	2	3	2	53	0.700
Tightened, 5	0	5	0	7	0.368

High negative-class accuracy therefore coexists with poor affected-guard recall. Pooled macro-F1 of 0.7211 is higher than stratified macro-F1 because aggregation weights the large number of easy unaffected pairs more heavily.

3.3 Failure modes

Table 2 decomposes all 14 false negatives. Tightened prohibitions account for eight: three in fan-out-one strata and five in the sole fan-out-five case. Relaxed permissions account for four and conditional exceptions for two.

The high-fan-out transition tightens confirmation: after the system lists a consequential action, a bare "yes" no longer suffices; the user must restate the action and target. This single clause affects profile update, cancellation, order modification, return, and exchange guards. All five affected artifacts score between 0.0689 and 0.0960, far below the frozen 0.20 threshold. The rewrite contains generic confirmation language, while each guard description emphasizes its local operation. IDF weighting cannot bridge an unshared behavioral relation.

This case illustrates, but does not causally prove, a distributed-dependency limitation. Fan-out, wording, confirmation topic, source clause, and tightened-prohibition class are fully confounded. A defensible conclusion is narrower: the lexical model missed every dependency in one cross-cutting high-fan-out case, and the overall packet showed additional misses across all three shift classes. Establishing a general fan-out scaling law would require multiple independently sampled clauses at each fan-out.

4 System Design Implications

The confirmatory failure changes the role that an impact predictor may safely play. We derive four requirements for policy-governed agent infrastructure.

Treat impact prediction as scheduling, not authorization. A predictor may rank guards for recompilation, prewarm caches, or focus human review. It must not conclude that an unselected guard remains valid. The 14 false negatives show that such a conclusion is unsupported even when development macro-F1 is near one.

Bind compiled artifacts to exact policy revisions. Every guard should carry immutable identifiers for the policy clauses and other inputs from which it was built. Before a protected action, the runtime should compare those identifiers with authoritative current revisions. This parallels derivation-currency systems that retain exact input links and block when evidence is unavailable (Chen et al., 2026). Our experiment does not test such a runtime; it motivates the requirement by showing that inferred dependencies are incomplete.

Keep the action-boundary check independent. Generated tool guards provide deterministic enforcement around invocations (Zwerdling et al., 2025), while policy-aware benchmarks show why completion alone is insufficient (Levy et al., 2026). The authoritative check should be implemented independently of the heuristic that selected artifacts for refresh. A revision mismatch, missing provenance, or ambiguous mapping should trigger fail-closed recompilation or review rather than heuristic release.

Validate in layers and preserve negative evidence. A practical release pipeline should separately test data provenance, impact-map coverage, guard behavior, and end-to-end action outcomes. Stage gates should freeze configurations before held-out evaluation and retain failed confirmatory results. The private decrease in this study is useful precisely because it was not converted into another tuning round.

These requirements constitute an architecture proposal, not a measured claim that layering eliminates unsafe commits. We did not run a live agent, execute generated guards, measure retry success, or estimate latency and token overhead. Those evaluations are necessary before making system-level safety or efficiency claims.

5 Related Work

τ-bench evaluates agents that interact with simulated users and domain APIs while following airline and retail policies, and reports substantial reliability degradation over repeated trials (Yao et al., 2024). We use frozen policy snapshots from its two domains, but evaluate policy-revision impact on compiled artifacts rather than conversational task completion.

ToolGuard compiles policy documents into tool-associated deterministic validators and places them at tool invocation boundaries (Zwerdling et al., 2025). Our study addresses the maintenance problem created when the source policy changes after guards exist. ST-WebAgentBench attaches explicit organizational, user, and task policies to web workflows and distinguishes completion under policy from nominal completion (Levy et al., 2026); our pairwise task similarly prevents unaffected-class prevalence from hiding affected-guard misses.

PlanFence distinguishes current observations from plans derived from current inputs and validates exact parent versions before protected actions (Chen et al., 2026). Its recorded-input contract supports the provenance layer we recommend. RuleDiff asks an earlier and less structured question—which artifacts might depend on a changed natural-language clause—and the private failure shows why inferred lexical links cannot substitute for recorded dependencies.

6 Limitations and Ethics

The development and private packets are small: eight and six source clauses. The retail labels were produced through two isolated policy-reading exercises followed by adjudication, but inter-annotator agreement was not quantified. The single high-fan-out transition prevents causal inference about fan-out. Domain, phrasing, and artifact style shift together. The 12 development runs are adaptive configurations rather than statistical replications, and no variance over random seeds applies to the deterministic model.

The benchmark labels indicate whether an artifact's expected behavior changes. They do not measure executable guard correctness, task completion, real policy violations, or harm. The term stale false negative must therefore not be interpreted as an unsafe production event. Conversely, the absence of observed execution harm cannot justify deployment because no live execution was evaluated.

The data consist of public policy text and synthetic annotations, with no observed user records or credentials. Releasing the terminal private labels enables verification but permanently prevents reuse of this packet as a blind test. Post-release improvements require a new held-out domain or independently constructed packet.

7 Reproducibility Statement

The artifact contains the complete 12-run ledger, machine-readable configurations, development and terminal private aggregates, frozen inputs and labels, source snapshots, unit tests, an independent aggregate recomputation, and SHA-256 checksums. It requires Python 3.11 or newer and no third-party packages or network access. A single command verifies all recorded development decisions and recomputes the G3 score, strata, errors, and seeded bootstrap interval. The one-shot timestamped result remains immutable; post-unblinding recomputation is identified as artifact validation. Third-party tau-bench and ToolGuard materials retain their MIT and Apache-2.0 notices, respectively.

8 Conclusion

A lexical policy-impact model that appeared nearly solved on development failed to transfer at that level: stratified macro-F1 decreased from 0.9919 to 0.6661 on a single frozen retail confirmatory packet. The 14 false negatives, including all five affected guards in one cross-cutting tightened-confirmation case, show that selective invalidation cannot be trusted as an authorization mechanism. The appropriate systems response is not to hide the negative result or retune on it, but to demote prediction to an optimization and retain provenance-backed, independently enforced, fail-closed checks at protected actions.

References

Chen, E., Wang, S., and Brinton, C. G. Fresh memory, stale plans: Derivation currency for distributed LLM-agent memory, 2026. URL https://arxiv.org/abs/2609.03340.

Levy, I., Wiesel, B., Marreed, S., Oved, A., Yaeli, A., Mashkif, N., and Shlomov, S. ST-WebAgentBench: A benchmark for evaluating safety and trustworthiness in web agents. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2410.06703.

Yao, S., Shinn, N., Razavi, P., and Narasimhan, K. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL https://arxiv.org/abs/2406.12045.

Zwerdling, N., Boaz, D., Rabinovich, E., Uziel, G., Amid, D., and Anaby-Tavor, A. Towards enforcing company policy adherence in agentic workflows. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track, 2025. doi: 10.18653/v1/2025.emnlp-industry.41. URL https://doi.org/10.18653/v1/2025.emnlp-industry.41.